Predicting NYC Taxi Fare Amount with Machine Learning

What is this notebook about?

In one sentence: *Can we predict a taxi fare from **passengers, distance and trip duration** before the trip ends?*

Audience: Beginners - every step explains *what*, *why*, and *how to read the output*.

Business Goal

Help drivers and apps **estimate fares upfront** for transparent pricing and revenue planning.

Dataset: `Trip_data.csv`

- **6,405,008 NYC trips, Jan 2020**, 18 columns
- Target: `fare_amount ($)` | Features: `passenger_count`, `trip_distance (miles)`, `time_duration (mins, engineered)`
- Payment columns dropped here - focus is trip physics, not payment method (see companion notebook *Maximum Revenue* for Card vs Cash).

Models Compared

1. **Linear Regression** - straight-line baseline, fast & interpretable
2. **Decision Tree** - if-then rules, captures non-linearity
3. **Random Forest** - many trees averaged, most robust

How to read

1. **Steps 2-8:** same rigorous cleaning as companion project -> **2.08M** modelling-ready trips
2. **Step 9:** 80/20 train-test split
3. **Steps 10-12:** train -> predict -> score (RMSE / R2 / MAE explained simply)
4. **Conclusion:** winner + business use

Metrics in plain English

- **R2 Accuracy %** - % of fare variation explained (higher = better, 100% perfect)
- **RMSE ($)** - typical error penalizing big misses (lower = better)
- **MAE ($)** - average absolute $ error (lower = better, easiest to explain)

Step 1 - Importing Libraries

`pandas` for tables, `matplotlib`/`seaborn` for checks, `sklearn` for splitting, training and scoring.

```
# Step 1: Import libraries
# pandas for tables, matplotlib/seaborn for plots, sklearn for modelling
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as st
import warnings
warnings.filterwarnings('ignore')
```

Step 2 - Loading the Data

Load the 6.4M-row CSV into `df`. This is the raw, messy real-world table.

```
# Step 2: Load raw NYC taxi data (6.4M trips, Jan 2020)
df = pd.read_csv('Trip_data.csv')
```

Step 3 - First Exploration

Always look before cleaning: sample rows, types, and the new duration feature.

```
# First look: 5 sample rows, 18 columns
df.head()
```

```
   VendorID tpep_pickup_datetime tpep_dropoff_datetime  passenger_count  \
0       1.0  2020-01-01 00:28:15   2020-01-01 00:33:03              1.0
1       1.0  2020-01-01 00:35:39   2020-01-01 00:43:04              1.0
2       1.0  2020-01-01 00:47:41   2020-01-01 00:53:52              1.0
3       1.0  2020-01-01 00:55:23   2020-01-01 01:00:14              1.0
4       2.0  2020-01-01 00:01:58   2020-01-01 00:04:16              1.0

   trip_distance  RatecodeID store_and_fwd_flag  PULocationID  DOLocationID  \
0            1.2         1.0                  N           238           239
1            1.2         1.0                  N           239           238
2            0.6         1.0                  N           238           238
3            0.8         1.0                  N           238           151
4            0.0         1.0                  N           193           193

   payment_type  fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
0           1.0          6.0    3.0      0.5        1.47           0.0
1           1.0          7.0    3.0      0.5        1.50           0.0
2           1.0          6.0    3.0      0.5        1.00           0.0
3           1.0          5.5    0.5      0.5        1.36           0.0
4           2.0          3.5    0.5      0.5        0.00           0.0

   improvement_surcharge  total_amount  congestion_surcharge
0                    0.3         11.27                   2.5
1                    0.3         12.30                   2.5
2                    0.3         10.80                   2.5
3                    0.3          8.16                   0.0
4                    0.3          4.80                   0.0
```

What you see: 5 trips with vendor, times, distance, fare, surcharges. Last rows of full data contain NaNs - handled next.

```
# Check column types - datetimes are text, need conversion
df.dtypes
```

```
VendorID                float64
tpep_pickup_datetime        str
tpep_dropoff_datetime       str
passenger_count         float64
trip_distance           float64
RatecodeID              float64
store_and_fwd_flag          str
PULocationID              int64
DOLocationID              int64
payment_type            float64
fare_amount             float64
extra                   float64
mta_tax                 float64
tip_amount              float64
tolls_amount            float64
improvement_surcharge   float64
total_amount            float64
congestion_surcharge    float64
dtype: object
```

Why: Timestamps are text (`object`). Must become datetime to compute duration. Numerics are float due to NaNs.

```
# Convert pickup/drop-off strings to real datetimes so we can compute duration
df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'])
df['tpep_dropoff_datetime'] = pd.to_datetime(df['tpep_dropoff_datetime'])
```

Done: Both timestamps are now true datetimes.

Feature Engineering: `time_duration` (minutes)

Fare depends on time as well as distance (traffic!). Formula: `(drop-off − pickup)` in minutes.

```
# Feature engineering: trip duration in minutes
df['time_duration'] = df['tpep_dropoff_datetime'] - df['tpep_pickup_datetime']
df['time_duration'] = df['time_duration'].dt.total_seconds()/60
```

```
# Preview table with new time_duration column
df.head()
```

```
   VendorID tpep_pickup_datetime tpep_dropoff_datetime  passenger_count  \
0       1.0  2020-01-01 00:28:15   2020-01-01 00:33:03              1.0
1       1.0  2020-01-01 00:35:39   2020-01-01 00:43:04              1.0
2       1.0  2020-01-01 00:47:41   2020-01-01 00:53:52              1.0
3       1.0  2020-01-01 00:55:23   2020-01-01 01:00:14              1.0
4       2.0  2020-01-01 00:01:58   2020-01-01 00:04:16              1.0

   trip_distance  RatecodeID store_and_fwd_flag  PULocationID  DOLocationID  \
0            1.2         1.0                  N           238           239
1            1.2         1.0                  N           239           238
2            0.6         1.0                  N           238           238
3            0.8         1.0                  N           238           151
4            0.0         1.0                  N           193           193

   payment_type  fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
0           1.0          6.0    3.0      0.5        1.47           0.0
1           1.0          7.0    3.0      0.5        1.50           0.0
2           1.0          6.0    3.0      0.5        1.00           0.0
3           1.0          5.5    0.5      0.5        1.36           0.0
4           2.0          3.5    0.5      0.5        0.00           0.0

   improvement_surcharge  total_amount  congestion_surcharge  time_duration
0                    0.3         11.27                   2.5       4.800000
1                    0.3         12.30                   2.5       7.416667
2                    0.3         10.80                   2.5       6.183333
3                    0.3          8.16                   0.0       4.850000
4                    0.3          4.80                   0.0       2.300000
```

Check: New `time_duration` column appears (e.g., 4.8 mins). Negative/zero values are clock errors - removed later.

Focus columns

Keep only what predicts fare: passengers, distance, duration + target fare. Dropping 14 unused cols speeds training.

```
# Keep only predictors + target: passengers, distance, duration -> fare
# (payment_type excluded here; focus is on trip physics)
df = df[['passenger_count','trip_distance','fare_amount','time_duration']]
```

```
# Preview focused 4-column table
df.head()
```

```
   passenger_count  trip_distance  fare_amount  time_duration
0              1.0            1.2          6.0       4.800000
1              1.0            1.2          7.0       7.416667
2              1.0            0.6          6.0       6.183333
3              1.0            0.8          5.5       4.850000
4              1.0            0.0          3.5       2.300000
```

Step 4 - Missing Values

Real data has gaps. 65,441 rows (~1%) lack `passenger_count`. With 6.4M rows, dropping is safe and unbiased.

```
# Count missing values per column
df.isnull().sum()
```

```
passenger_count    65441
trip_distance          0
fare_amount            0
time_duration          0
dtype: int64
```

Reading: Only `passenger_count` has NaNs; distance/fare/duration complete.

```
# Drop rows with missing passenger_count (65,441 rows, ~1%)
df.dropna(inplace=True)
```

```
# Convert passenger_count to int for modelling
df['passenger_count'] = df['passenger_count'].astype('int64')
```

Done: No NaNs left; `passenger_count` is now integer.

Step 5 - Duplicates

Identical rows would let the model memorize rather than learn. We found **~3.76M** duplicates (common short-trip patterns).

```
# List exact duplicate rows
df[df.duplicated()]
```

```
         passenger_count  trip_distance  fare_amount  time_duration
1032                   1           1.11          6.0       5.700000
1409                   1           3.00         12.0      12.683333
1754                   1           0.80          5.0       4.016667
2056                   1           0.00          7.0       0.000000
2095                   1           0.00          2.5       0.066667
...                  ...            ...          ...            ...
6339559                1           1.81          8.5       8.016667
6339560                1           0.98          6.5       6.900000
6339562                1           2.10         11.0      14.233333
6339565                1           1.61          8.5       9.633333
6339566                1           0.00          0.0       1.066667

[3763798 rows x 4 columns]
```

```
# Drop duplicates to avoid training on repeated patterns
df.drop_duplicates(inplace = True)
```

```
# After dedup: ~2,575,769 unique rows
df
```

```
         passenger_count  trip_distance  fare_amount  time_duration
0                      1           1.20          6.0       4.800000
1                      1           1.20          7.0       7.416667
2                      1           0.60          6.0       6.183333
3                      1           0.80          5.5       4.850000
4                      1           0.00          3.5       2.300000
...                  ...            ...          ...            ...
6339546                1          10.45         36.0      36.666667
```

```
6339555                3           2.09         10.0      14.800000
6339561                1           4.11         17.5      21.500000
6339563                1           2.13         13.0      19.000000
6339564                1           2.55         12.5      16.283333

[2575769 rows x 4 columns]
```

After dedup: ~2,575,769 unique trips remain.

Step 6 - Filter Invalid Passengers

56.5% solo, 19% duo. `0` passengers impossible; `7/8/9` ultra-rare errors. Keep 1-6 for realistic training.

```
# Solo riders dominate (56.5%); 0 and 7/8/9 are invalid/rare
df['passenger_count'].value_counts(normalize=True)
```

```
passenger_count
1    0.565451
2    0.191530
3    0.070259
5    0.067651
6    0.043234
4    0.039102
0    0.022748
7    0.000011
9    0.000007
8    0.000007
Name: proportion, dtype: float64
```

```
# Keep realistic groups 1-6 passengers only
df = df[(df['passenger_count'] > 0) & (df['passenger_count'] < 7)]
```

Step 7 - Fix Impossible Values & Outliers

`describe()` shows negatives: −22 miles, −$1238 fare, −2770 mins - clearly sensor/entry errors.

```
# Stats BEFORE removing impossible values - note negatives!
df.describe()
```

```
       passenger_count  trip_distance   fare_amount  time_duration
count     2.517113e+06   2.517113e+06  2.517113e+06   2.517113e+06
mean      1.957943e+00   4.881218e+00  1.874048e+01   2.601713e+01
std       1.462778e+00   5.090283e+00  1.582014e+01   9.970095e+01
min       1.000000e+00  -2.218000e+01 -1.238000e+03  -2.770367e+03
25%       1.000000e+00   1.610000e+00  9.500000e+00   1.050000e+01
50%       1.000000e+00   3.000000e+00  1.400000e+01   1.665000e+01
75%       2.000000e+00   6.100000e+00  2.250000e+01   2.455000e+01
max       6.000000e+00   2.628800e+02  4.265000e+03   8.525117e+03
```

```
# Remove impossible trips: distance, fare, duration must be > 0
df = df[df['trip_distance'] > 0]
df = df[df['fare_amount']>0]
df = df[df['time_duration']> 0]
```

```
# Stats AFTER >0 filter: mins sensible, max still huge (outliers)
df.describe()
```

```
       passenger_count  trip_distance   fare_amount  time_duration
count     2.472789e+06   2.472789e+06  2.472789e+06   2.472789e+06
mean      1.967843e+00   4.952124e+00  1.876515e+01   2.613965e+01
std       1.468404e+00   5.091919e+00  1.502664e+01   1.000032e+02
```

```
min       1.000000e+00   1.000000e-02  1.000000e-02   1.666667e-02
25%       1.000000e+00   1.670000e+00  9.500000e+00   1.065000e+01
50%       1.000000e+00   3.050000e+00  1.400000e+01   1.671667e+01
75%       2.000000e+00   6.200000e+00  2.250000e+01   2.455000e+01
max       6.000000e+00   2.628800e+02  4.265000e+03   8.525117e+03
```

After `>0` filter: ~2.47M rows; minimums sensible. Max still wild ($4265, 262 miles) -> need IQR.

IQR Outlier Removal (simple)

Take middle 50% (Q1-Q3). Keep only `Q1−1.5×IQR` to `Q3+1.5×IQR`. Removes extreme luxury/long trips that would skew the line.

```
# IQR outlier removal for fare, distance, duration (Q1-1.5*IQR to Q3+1.5*IQR)
for col in ['fare_amount','trip_distance','time_duration']:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)

    iqr = q3-q1

    lower_bound = q1-(1.5*iqr)
    upper_bound = q3 + (1.5*iqr)

    df = df[(df[col]>= lower_bound) & (df[col]<= upper_bound)]
```

```
# Final modelling dataset: ~2,082,993 clean trips
df
```

```
         passenger_count  trip_distance  fare_amount  time_duration
0                      1           1.20          6.0       4.800000
1                      1           1.20          7.0       7.416667
2                      1           0.60          6.0       6.183333
3                      1           0.80          5.5       4.850000
5                      1           0.03          2.5       0.883333
...                  ...            ...          ...            ...
6339534                2           1.20          9.0      13.516667
6339555                3           2.09         10.0      14.800000
6339561                1           4.11         17.5      21.500000
6339563                1           2.13         13.0      19.000000
6339564                1           2.55         12.5      16.283333

[2082993 rows x 4 columns]
```

Final modelling data: 2,082,993 clean trips. Balanced, realistic, ready to split.

Step 9 - Train-Test Split (80/20)

Why: Train on 80% (~1.66M), test on unseen 20% (~416k) to measure real-world performance. `random_state=42` makes split reproducible.

```
# Train-test split utility: 80% train, 20% test
from sklearn.model_selection import train_test_split
```

```
# Split features (X) and target fare (y). random_state=42 makes it reproducible.
X_train,                    X_test,                    y_train,                    y_test                    =
train_test_split(df.drop('fare_amount',axis=1),df['fare_amount'],test_size=0.2,random_state=42)
```

Result: `X_train/X_test` = 3 features, `y_train/y_test` = fare. No leakage - model never sees test fares during training.

Step 10 - Fare Amount Prediction

We trial 3 algorithms from simplest to most powerful. Same data, same split - fair comparison.

1. Linear Regression
2. Decision Tree Regressor
3. Random Forest Regressor

Model 1 - Linear Regression

Fits `fare = w1×passengers + w2×distance + w3×duration + bias`. Assumes roughly linear relationship - true for taxis (longer/farther ≈ pricier). Fast, interpretable baseline.

```
# Model 1: Linear Regression - fits a straight-line formula fare = w1*passengers + w2*distance + w3*duration + b
from sklearn.linear_model import LinearRegression
```

```
# Create and train the linear model on training data
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression()
```

```
# Predict fares for unseen test trips
prediction = model.predict(X_test)
```

```
# Error metrics: MAE (avg $ error), RMSE (penalizes big errors), R2 ( % variance explained)
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
```

```
# Compute RMSE for linear regression
# Checking the model accuracy
lin_reg_acc = root_mean_squared_error(prediction, y_test)
```

```
# Show linear regression scores
print("Root mean squared error:",lin_reg_acc )
print(f"Accuracy: {r2_score(y_test, prediction)*100:.2f}%")
print("MAE:", mean_absolute_error(y_test, prediction))
```

```
Root mean squared error: 0.9496492626833282
Accuracy: 97.87%
MAE: 0.4912443737823128
```

Model 2 - Decision Tree Regressor

Series of yes/no splits - e.g., `if distance > 5 AND duration > 20 -> $28`. Captures curves and interactions linear model misses, but single tree can overfit (memorize quirks).

```
# Model 2: Decision Tree - learns if-then splits (e.g. if distance > 5 miles and duration > 20 mins...)
from sklearn.tree import DecisionTreeRegressor
```

```
# Create tree with fixed seed for reproducibility
dt = DecisionTreeRegressor(random_state=42)
```

```
# Train tree on same training data
dt.fit(X_train, y_train)
```

```
DecisionTreeRegressor(random_state=42)
```

```
# Predict test fares with the tree
prediction = dt.predict(X_test)
```

```
# Show decision-tree scores
```

```
print("Root mean squared error:",root_mean_squared_error(y_test, prediction) )
print(f"Accuracy: {r2_score(y_test, prediction)*100:.2f}%")
print("MAE:", mean_absolute_error(y_test, prediction))
```

```
Root mean squared error: 1.2114006808745843
Accuracy: 96.53%
MAE: 0.5823368474468825
```

Model 3 - Random Forest Regressor

Grow 100 different trees on random subsets and average them. Averaging cancels individual overfitting while keeping non-linear power. Slower to train, usually strongest.

```
# Model 3: Random Forest - averages many trees to reduce overfitting
from sklearn.ensemble import RandomForestRegressor
```

```
# Create forest (default 100 trees) with fixed seed
rfc = RandomForestRegressor(random_state=42)
```

```
# Train forest - slower but more robust
rfc.fit(X_train, y_train)
```

```
RandomForestRegressor(random_state=42)
```

```
# Predict test fares with the forest
prediction = rfc.predict(X_test)
```

```
# Show random-forest scores
print("Root mean squared error:",root_mean_squared_error(y_test, prediction) )
print(f"Accuracy: {r2_score(y_test, prediction)*100:.2f}%")
print("MAE:", mean_absolute_error(y_test, prediction))
```

```
Root mean squared error: 0.9968786355607281
Accuracy: 97.65%
MAE: 0.49454855719433904
```

Conclusion & Recommendation

Winner: Linear Regression

Linear Regression achieved the highest R2 score and the lowest RMSE among the three models. It also has a simple, interpretable structure, making it useful for estimating taxi fares.

Business Use

- **Upfront estimates:** Provide estimated fares to passengers before a trip.
- **Driver planning:** Estimate fares for different trip distances and durations.
- **Easy deployment:** Use the trained model to generate predictions within an application.